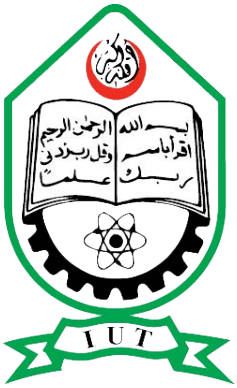
Islamic University of Technology



Machine Learning

CSE 4621

Universal Approximation Theorem

Author:
Ishmam Tashdeed
Lecturer
IUT, CSE

Contents

1 Why Can Neural Networks Learn Anything?

You have probably heard that neural networks are powerful enough to learn almost any pattern from data. But *why* is that true? Is it just hype, or is there actual mathematics behind it?

The **Universal Approximation Theorem (UAT)** is the mathematical answer. In plain terms, it says:

Key Takeaway

A neural network with even *one* hidden layer (but enough neurons) can approximate **any** reasonable function to any desired level of accuracy.

Think of it this way: a function is just a machine that takes some input and produces an output. The UAT tells us that a neural network can mimic *any such machine*, as closely as we want, given enough neurons.

2 What Is a Neural Network?

2.1 A Single Neuron

A single neuron takes a bunch of numbers $x_1, x_2, \ldots, x_n$, multiplies each by a weight, adds a bias, and then passes the result through an **activation function** σ:

$$\text{output} = \sigma\left(\sum_{j=1}^{n} w_j x_j + b\right)$$

where w_j are the weights and b is the bias. The activation function introduces *non-linearity*, which is the key ingredient.

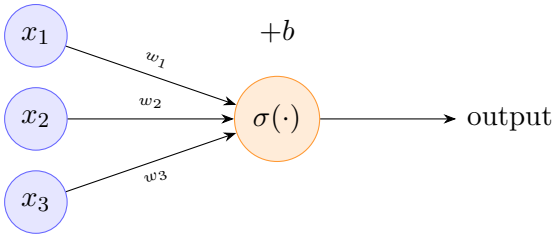


Figure 1: A single neuron. Inputs are weighted, summed, shifted by bias b, then passed through activation σ.

2.2 A Multi-Layer Perceptron (MLP)

Stack neurons into **layers**: one input layer, one or more hidden layers, and one output layer. Signals flow forward from input to output.

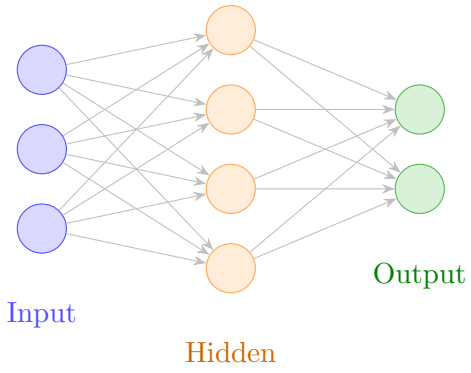


Figure 2: A simple MLP with one hidden layer. Every neuron in a layer is connected to every neuron in the next.

Mathematically, the output of a one-hidden-layer network with N neurons is:

$$f(\mathbf{x}) = \sum_{k=1}^{N} c_k \, \sigma\big(\mathbf{w}_k^\top \mathbf{x} + b_k\big)$$

where $\mathbf{w}_k$ are weight vectors, b_k are biases, and c_k are output weights.

3 The Universal Approximation Theorem

3.1 What Functions Are We Trying to Approximate?

We focus on *continuous functions* on a compact (closed and bounded) set. Think of something like:

$$g : [0,1]^n \to \mathbb{R}$$

a smooth or even just continuous mapping from n-dimensional input to a real output.

3.2 What Activation Functions Work?

The theorem requires the activation function σ to be **non-constant** and **continuous**. The classic choices are:

- **Sigmoid:** $\sigma(z) = \dfrac{1}{1+e^{-z}}$
- **Tanh:** $\sigma(z) = \tanh(z)$
- **ReLU:** $\sigma(z) = \max(0, z)$

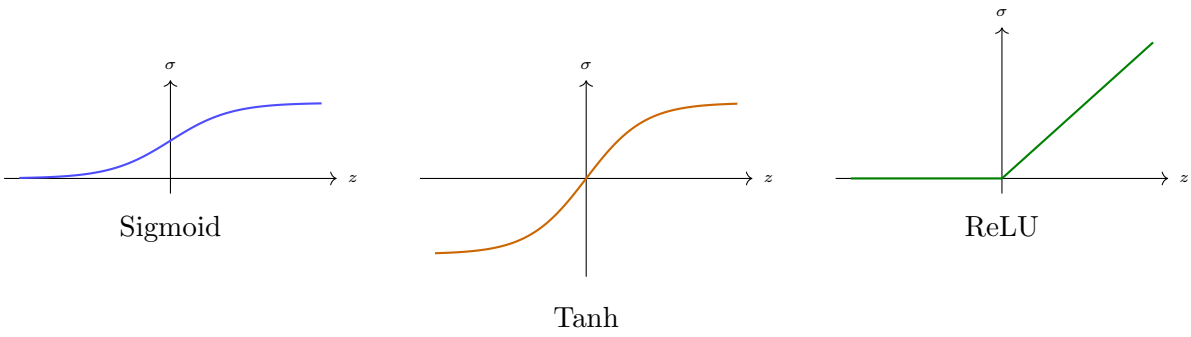


Figure 3: Three common activation functions. All are non-constant and continuous (ReLU is continuous but not differentiable at 0).

3.3 The Theorem

Theorem

Universal Approximation Theorem (Cybenko 1989; Hornik 1991): Let σ be any continuous, non-constant, bounded activation function (e.g. sigmoid or tanh). Let $g : [0,1]^n \to \mathbb{R}$ be any continuous function. Then for any $\varepsilon > 0$, there exist an integer N and parameters $\{c_k, \mathbf{w}_k, b_k\}_{k=1}^{N}$ such that the network

$$f(\mathbf{x}) = \sum_{k=1}^{N} c_k \, \sigma\big(\mathbf{w}_k^\top \mathbf{x} + b_k\big)$$

satisfies

$$\sup_{\mathbf{x} \in [0,1]^n} |f(\mathbf{x}) - g(\mathbf{x})| < \varepsilon.$$

Note

In plain English: no matter how small you set the error budget ε, there always *exists* a one-hidden-layer network that stays within ε of the target function g everywhere on the input domain. We may need many neurons N, but finitely many will always suffice.

4 How Does It Actually Approximate?

4.1 Building Bumps with Neurons

Here is the key geometric insight. Each neuron with a sigmoid-like activation carves out a smooth "step" or "bump" in the function. By combining many bumps with different positions, widths, and heights, we can approximate any shape.

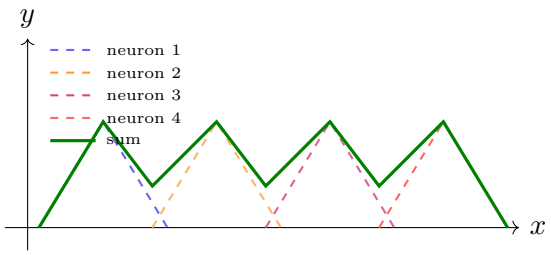


Figure 4: Each neuron contributes a localized bump (dashed). Their weighted sum (solid green) is the network's output. More neurons $\Rightarrow$ finer coverage $\Rightarrow$ better approximation.

4.2 What Happens as We Add More Neurons?

As N increases, the bumps become narrower and more numerous, filling in the gaps. The approximation error shrinks.

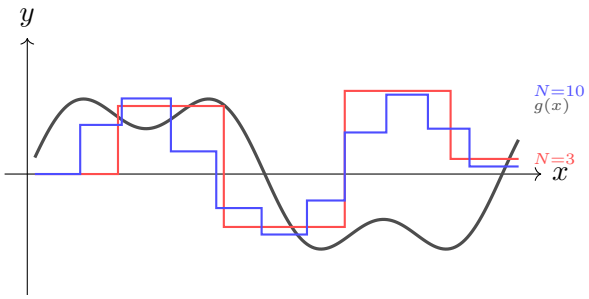


Figure 5: As N grows, the network output (coloured) approaches the true function $g(x)$ (black). Larger $N \Rightarrow$ smaller error.

5 What the Theorem Does *Not* Say

Note

The UAT is an *existence* result. It tells us the right parameters **exist**, but says nothing about:

- **How to find them** – training (gradient descent) is a separate story.
- **How many neurons are needed** – N could be astronomically large for a complicated g.
- **Generalization** – approximating training data well does not mean the network will do well on unseen data.
- **Depth** – the theorem covers one hidden layer, but in practice *deep* networks are far more efficient.

6 Why Depth Helps (A Quick Note)

The UAT guarantees width (many neurons in one layer) is enough. But using *many layers* (depth) often requires *exponentially fewer* total neurons for the same accuracy. Think of it as the difference between building a complex shape out of one thick slab vs. stacking many thin, precisely cut sheets.

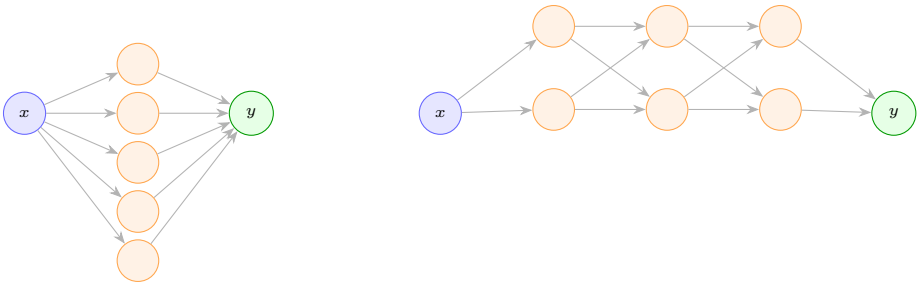


Figure 6: Shallow vs. deep networks. Both can approximate the same function, but depth achieves it with far fewer total parameters in many practical cases.

7 Summary

Key Takeaway

- A neural network with **one hidden layer** and enough neurons can approximate *any* continuous function on a bounded domain to any desired accuracy $\varepsilon > 0$.
- The key ingredient is a **non-linear activation function** – without it, stacking layers gives nothing more than a linear model.
- The theorem is an **existence guarantee**, not a recipe; it does not tell us how to train the network or how many neurons we will need.
- In practice, **depth** (more layers) is usually more efficient than width (more neurons per layer).